

Computer Systems and Professional Practice Solutions

Main Summer Examinations 2025

Note

Answer ALL questions.

Question 1

- (a) Explain why it is desirable to have more than one representation for a value. Your answer should provide an example of where this might be useful. **[3 marks]**
- (b) (i) Convert 118_{10} to an 8-bit unsigned binary number. **[3 marks]**
 (ii) Convert 1111101001101011_2 to hexadecimal. **[3 marks]**
 (iii) Show how -44_{10} can be represented using two's complement. **[3 marks]**
 (iv) Show how 42_{10} can be subtracted from 56_{10} in two's complement. **[3 marks]**
 (v) Using an example, explain the concept of underflow in binary. **[5 marks]**
- (c) Combinatorial logic gates can be used to implement Boolean expressions.
- (i) Show how the EX-OR function can be implemented using only NAND gates. Your answer should provide a Boolean expression for the EX-OR function. **[5 marks]**
- (ii) Show the truth table and logic circuit for a 1-bit Full-Adder. **[5 marks]**
- (d) D-type flip-flops are commonly used in circuit design.
- (i) Draw and explain the truth table for a D-type flip flop. **[5 marks]**
- (ii) Show how D-type flip flops can be used in the design of an N-bit shift register. Your design should be serial-in, serial-out. **[5 marks]**

Model answer / LOs / Creativity:

- (a) Having more than one representation for a value is desirable because different representations are optimized for different operations and applications, enabling efficient computation, storage, and interpretation. In computer systems, this flexibility enhances performance, compatibility, and ease of processing.

For example, integer values can be represented in multiple ways such as unsigned binary, signed magnitude, two's complement, and BCD (Binary-Coded Decimal). In arithmetic operations, two's complement is preferred because it simplifies the hardware design for addition and subtraction, as the same circuitry can be used for both positive and negative numbers. Conversely, BCD is useful in financial and business applications where decimal accuracy is required, as it directly represents each decimal digit in binary form, preventing rounding errors common in floating-point representations.

- (i) 01110110_2
- (ii) $FA6B_{16}$
- (iii) 11010100_{TC}
- (iv) 00001110_2
- (v) Underflow occurs when a calculation produces a result smaller than the minimum value representable by the given number of bits. In binary arithmetic, this typically happens in fixed-size registers or memory locations when subtracting a larger number from a smaller one using two's complement. Any example, under any number system, demonstrating the concept is acceptable.

(b) The EX-OR function should be expressed as:

$$A \oplus B = A \cdot \overline{B} + \overline{A} \cdot B$$

Any correct NAND-only schematic will gain full credit. The NAND-only Boolean expression is not required for full credit.

- (c) A 1-bit Full Adder adds three binary digits: two operands (A and B) and a carry-in (C_{in}). It produces a sum (S) and a carry-out (C_{out}).

Table 1: Truth table for a 1-bit Full Adder.

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

- (d) (i) A D-type flip-flop captures the value of the data input (D) on the rising edge of the clock (CLK) and stores it until the next clock cycle. The output (Q) is updated to the input value, while the inverse output ($\overline{Q}$) is the opposite.
- (ii) A Serial-In Serial-Out (SISO) Shift Register shifts data through a series of flip-flops one bit at a time with each clock pulse. It takes a serial input and outputs the data in serial form after N clock cycles, where N is the number of bits in the register. N flip-flops are connected in series, where the output of one flip-flop is connected to the input of the next. All flip-flops are triggered by

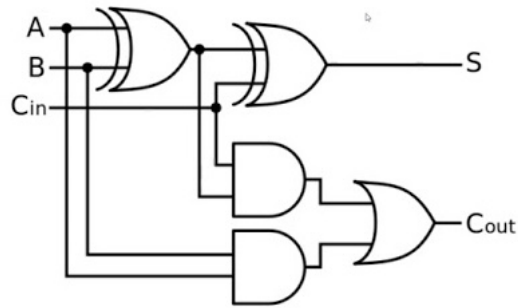


Figure 1: Logic circuit for a 1-bit Full Adder

Table 2: Truth table for a D-type flip-flop

CLK	D	Q	$\overline{Q}$
0	0	Q_{prev}	$\overline{Q_{prev}}$
0	1	Q_{prev}	$\overline{Q_{prev}}$
^	0	0	1
^	1	1	0
1	0	Q_{prev}	$\overline{Q_{prev}}$
1	1	Q_{prev}	$\overline{Q_{prev}}$

a common clock, ensuring synchronous data movement. Data is fed into the first flip-flop in the series. The output of the last flip-flop in the chain provides the serial output.

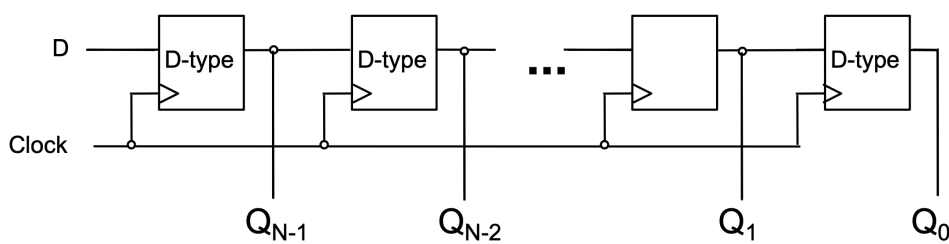


Figure 2: Logic circuit for an N-bit shift register based on D-type flip flops.

Learning outcomes: Explain common data representations and their role in computer systems. Demonstrate basic systems programming and digital circuit prototyping abilities.

Question 2

- (a) Explain the memory hierarchy using a labelled diagram. Your answer should focus on the motivation for the hierarchy and its role in computer system design. **[10 marks]**
- (b) Outline the implications of Moore's Law for the design of computer systems. Your answers should state Moore's Law. **[4 marks]**
- (c) Explain the operation of Dynamic RAM (DRAM). **[4 marks]**
- (d) Parity codes can be used to detect errors in message transmission.
- Explain how parity codes enable the detection of errors in message transmission. Your answer should provide an example that uses even parity. **[4 marks]**
 - Outline the disadvantages of using parity codes for the detection of errors in message transmission. **[4 marks]**
 - Show how parity codes for the detection of errors can be computed in hardware using combinatorial logic. **[4 marks]**

Model answer / LOs / Creativity:

- (a) Many factors influence the choice of memory technology, e.g., Frequency of access, access time, capacity required, cost. We may desire low cost and high capacity - leads to choice of cheap per bit, high capacity and slow access time. We may desire high performance - leads to choice of expensive per bit, low capacity and fast access time. To solve the problem we choose both and organise into a hierarchy. Economics (cost per bit) is typically viewed as a primary motivator. Developed discussion, potentially offering examples, is expected for full credit.

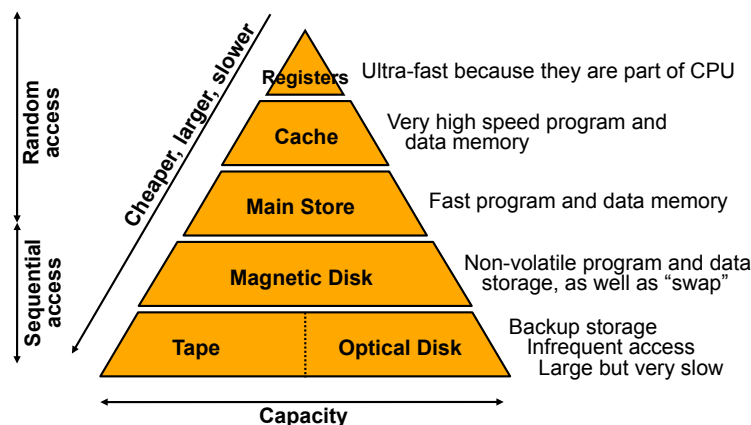


Figure 3: Diagram of the memory hierarchy

- (b) Moore's Law states that the number of transistors on a microchip doubles approximately every 18-months / two years, resulting in increased processing power and reduced cost per transistor. This exponential growth leads to continuous improvements in computing performance, efficiency, and miniaturisation. Doubling of transistors enhances CPU performance, enabling faster processing speeds and more complex calculations. The implication is that designers can create more powerful processors, supporting advanced applications like AI, machine learning, and real-time data processing. As more transistors are packed into a chip, the cost per transistor decreases. The implication is more affordable computing power, leading to cost-effective devices and broader accessibility of high-performance technology. Transistor shrinkage enables smaller, lighter, and more energy-efficient devices. The implication is portable computing devices such as smartphones, tablets, and wearables become more powerful and efficient. Increased transistor density generates more heat, requiring innovative cooling solutions. The implication is that designers must address power consumption and thermal management to maintain reliability and performance.
- (c) DRAM stores each bit of data in a capacitor and a transistor within a memory cell. The capacitor holds an electrical charge representing a binary '1' or '0': Charged = 1, Discharged = 0. The transistor acts as a switch, controlling the access to the capacitor. The capacitor gradually leaks charge, so the stored data would be lost without refreshing. DRAM requires periodic refresh cycles to maintain data integrity. The memory controller automatically refreshes each row at regular intervals (e.g., every 64 ms). Higher density and lower cost per bit compared to SRAM, hence it is suitable for main memory. Slower access speed than SRAM due to the need for refreshing. Increased power consumption due to periodic refresh cycles.
- (d) (i) Parity codes are error-detecting codes used to identify single-bit errors during message transmission. A parity bit is added to the data to ensure the total number of 1s is either even (even parity) or odd (odd parity).
Even parity ensures an even number of 1s in the entire data including the parity bit. The sender calculates the parity bit by counting the number of 1s in the data: If the number of 1s is even, the parity bit is 0. If the number of 1s is odd, the parity bit is 1 to make the total even. The receiver recalculates the parity upon receiving the message: If the total number of 1s is even, the data is assumed to be correct. If the total number of 1s is odd, an error is detected.
Example: Original 7-bit data is 1011001. Number of 1s = 4 (even). Parity bit = 0 (to maintain even parity). Transmitted data: 10110010. Assume a single-bit error occurs, changing the message to 10110110. Number of 1s = 5 (odd). The receiver counts the 1s and finds an odd number. This indicates an error. Parity codes can only detect an odd number of errors.
- (ii) Parity codes can only detect single-bit errors. They cannot detect multiple-bit errors if an even number of bits are flipped, as the parity would still appear

correct. For example, if two bits change in a message, the total number of 1s may still be even, leading to an undetected error. Parity codes cannot correct errors, only detect them. If an error is detected, the entire message must be retransmitted, leading to inefficiencies. The addition of a parity bit increases the data size by one bit for every set of data. This leads to increased bandwidth usage and reduced transmission efficiency, especially in systems transmitting large volumes of data. Parity checks are insufficient for systems requiring high data integrity, such as financial transactions or critical communication systems. More robust error-detection and correction methods, such as Hamming codes or Cyclic Redundancy Check (CRC), are preferred in such cases. Parity codes are generally used for simple binary data streams. They are inefficient for complex data structures or files with multiple layers of encoding (e.g., multimedia files).

- (iii) The basis for the implementation is a set of EX-OR gates connected in series to compute the parity bit.

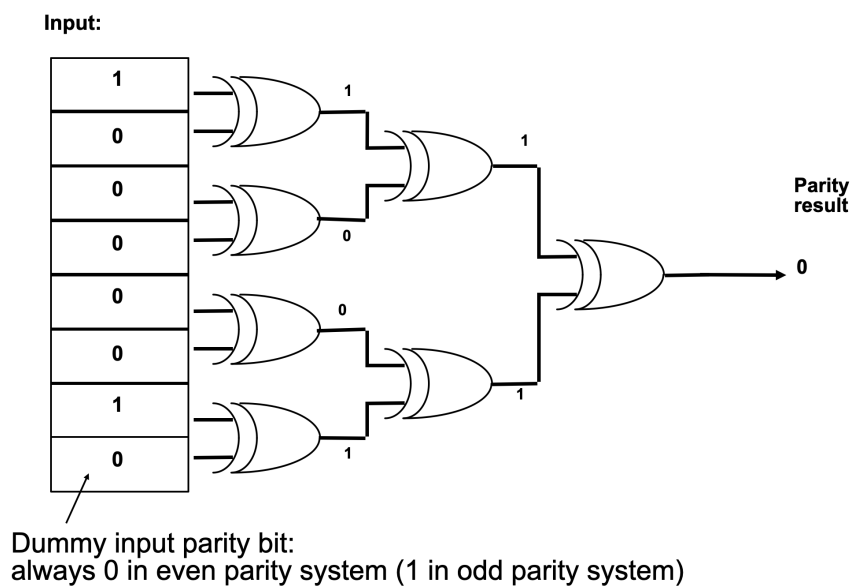


Figure 4: Parity code generator using EX-OR logic gates.

Learning outcomes: Demonstrate basic systems programming and digital circuit prototyping abilities. Discuss hardware organisation and architecture of computer systems and their components, including logic components, microprocessors and I/O systems. Discuss a range of legal, social, ethical and professional issues relating to the design, implementation and deployment of computer systems.

Question 3

- (a) Microprocessors execute instructions on the basis of a set of interacting components.

- (i) Provide a detailed explanation of the fetch-decode-execute cycle. Your answer should provide details of the microprocessor components involved. **[10 marks]**
 - (ii) Distinguish between micro-instructions and macro-instructions. **[4 marks]**
- (b) Control units can be implemented using a hardwired approach.
- (i) Explain the operation of hardwired control units. **[4 marks]**
 - (ii) Outline the advantages and disadvantages of adopting a hardwired approach to control unit design. **[4 marks]**
- (c) Interrupts are a commonly used I/O mechanism.
- (i) Explain the operation of interrupt-driven I/O. **[4 marks]**
 - (ii) Outline the advantages and disadvantages of interrupt-driven I/O. **[4 marks]**

Model answer / LOs / Creativity:

- (a) (i) The fetch-decode-execute cycle is the fundamental process by which a microprocessor retrieves and executes instructions from memory, coordinated by the Control Unit (CU). During the fetch phase, the Program Counter (PC), which holds the address of the next instruction, is sent to the Memory Address Register (MAR). The address is placed on the address bus, and the instruction is fetched from the memory into the Memory Data Register (MDR). The instruction is then transferred to the Instruction Register (IR), and the PC is incremented to point to the next instruction. In the decode phase, the Control Unit interprets the instruction stored in the IR by breaking down the opcode and identifying the required operation and operands. The CU generates control signals to coordinate the activities of other components such as the Arithmetic Logic Unit (ALU), registers, and memory, ensuring the correct sequence of operations. It directs data flow through buses and activates the relevant circuits for decoding. During the execute phase, the CU sends signals to trigger the operation specified by the instruction. This could involve arithmetic or logical operations in the ALU, data transfer between memory and registers, or control operations affecting the PC or flags in the Status Register. The CU manages timing and synchronisation to ensure each step is completed before the next begins. After execution, the cycle repeats with the next instruction, enabling continuous processing of program instructions under the coordination of the Control Unit.
- (ii) Micro-instructions are low-level, detailed instructions that control the internal operations of a microprocessor at the hardware level. They are stored in the control memory of the Control Unit and specify the exact sequence of control signals needed to perform elementary operations, such as moving data between

registers, performing arithmetic in the Arithmetic Logic Unit (ALU), or managing buses. Micro-instructions are typically used in microprogrammed control units, where each step of the fetch-decode-execute cycle is broken down into smaller operations. They directly control the hardware components and are not visible to programmers.

In contrast, macro-instructions are high-level instructions that are part of the computer's instruction set architecture, such as ADD, SUB, LOAD, or STORE. These instructions are written in assembly or machine language and are executed by the microprocessor. A single macro-instruction corresponds to a sequence of multiple micro-instructions. For example, an ADD macro-instruction might require a series of micro-instructions to fetch operands, perform the addition in the ALU, and store the result. Macro-instructions are visible to programmers and are translated by the Control Unit into the corresponding micro-instructions for execution. In summary, micro-instructions operate at the hardware level and control internal operations, while macro-instructions are high-level commands executed by the microprocessor as part of its instruction set.

- (b) (i) A hardwired control unit is a type of control unit used in microprocessors that generates control signals using fixed electronic circuits composed of combinational logic gates, flip-flops, and decoders. Unlike microprogrammed control units, which use stored microinstructions, hardwired control units use a pre-defined logic circuit to directly produce control signals for the execution of instructions.

During the fetch-decode-execute cycle, the Instruction Register (IR) holds the opcode of the current instruction. This opcode is sent to the Instruction Decoder, which decodes it into a unique set of signals corresponding to the specific operation. The decoded signals are then passed through a series of combinational logic circuits that generate the necessary control signals to coordinate data movement between components like the Arithmetic Logic Unit (ALU), registers, and memory. These signals control functions such as data routing on buses, activating read or write operations, and triggering arithmetic or logical operations in the ALU.

- (ii) Hardwired control units are extremely fast because control signals are generated through combinational logic circuits without the need for memory lookup or microinstruction sequencing. This results in faster instruction execution, making them ideal for high-performance processors, particularly in RISC (Reduced Instruction Set Computer) architectures. Hardwired designs require less space as they do not need control memory to store microinstructions, leading to more compact and cost-effective processors. Hardwired control units are straightforward to design and implement for processors with simple and fixed instruction sets, making them suitable for embedded systems and dedicated

hardware applications. Due to the absence of control memory and the use of efficient logic circuits, hardwired control units generally consume less power, which is beneficial for power-sensitive devices.

Hardwired control units are difficult to modify or update because any change to the instruction set or architecture requires redesigning the entire logic circuitry. This makes them unsuitable for processors that need to support evolving instruction sets or complex operations. For processors with complex instruction sets (CISC), designing hardwired control units becomes highly complicated, error-prone, and time-consuming due to the increased number of control signals and states. The complexity of designing and testing hardwired circuits increases development costs and time, particularly for complex processors. Troubleshooting and maintaining hardwired control units are challenging due to the lack of microinstructions, making it harder to trace and fix design errors.

- (c) (i) Interrupt-driven I/O is a mechanism that allows peripheral devices to signal the CPU when they require attention, enabling efficient communication between the processor and I/O devices without continuous polling. This approach enhances CPU efficiency by allowing it to execute other tasks until an I/O operation is needed.

During interrupt-driven I/O, when a device (e.g., keyboard, mouse, or network card) is ready for data transfer or needs processing, it sends an interrupt signal to the CPU through the Interrupt Request Line (IRQ). The CPU, upon receiving the interrupt, completes its current instruction and then temporarily suspends its operation by saving the Program Counter (PC) and the Processor Status Register onto the stack. It then executes an Interrupt Service Routine (ISR) corresponding to the specific device.

The Interrupt Controller identifies the source of the interrupt and prioritises it if multiple requests occur simultaneously. It then directs the CPU to the appropriate Interrupt Vector—a memory location containing the address of the ISR. The ISR executes the required operation, such as reading data from an input device or sending data to an output device. Once the ISR is completed, the CPU restores the saved state from the stack and resumes execution of the interrupted program.

- (ii) The CPU is not constantly polling devices, allowing it to execute other instructions until an interrupt occurs. This leads to better CPU utilisation and overall system efficiency, as processing power is used productively. Interrupts allow immediate attention to I/O events as they occur, ensuring lower latency and quicker response times compared to polling. Interrupt-driven I/O supports multiple devices more efficiently by using interrupt vectors and prioritisation schemes, making it easier to expand systems with additional peripherals. Since the CPU is not constantly checking devices, it can enter low-power states between interrupts, leading to reduced power consumption, which is beneficial for

portable and embedded systems.

Frequent interrupts can lead to interrupt overhead, where the CPU spends more time saving states and handling interrupts than executing the main program. This overhead can reduce system performance, particularly in real-time systems. Writing Interrupt Service Routines (ISRs) requires careful programming to ensure they are short, efficient, and do not conflict with other ISRs or processes. In systems with multiple devices, interrupt priority levels must be managed to prevent conflicts, leading to complex interrupt management. If interrupts are not properly managed, a device can generate a continuous stream of interrupts, causing an interrupt storm that overwhelms the CPU and halts normal operation.

Learning outcomes: Discuss hardware organisation and architecture of computer systems and their components, including logic components, microprocessors and I/O systems.